Anti-Fragile Agentic Workflow (AFAW)

A deterministic, multi-agent methodology for AI-assisted software development

Agustin Diaz-Cano

MS Candidate, Information Systems Engineering,
National Technological University (UTN), Argentina

ORCID: 0009-0001-4336-490X

DOI: 10.5281/zenodo.23050310

adiazcano@frba.utn.edu.ar

White paper • Version 1.1 • September 30, 2026

Abstract

Coding agents based on large language models can produce code faster than a person can review it. Running several of them on one repository adds failure modes that a single-agent workflow does not have: agents working on the wrong branch, collisions on shared context files, claims of results that were never measured, and tests that pass whatever the code does. This paper describes the Anti-Fragile Agentic Workflow (AFAW), a repository-level methodology and boilerplate for AI-assisted development with several agents in parallel. AFAW rests on one division of labour: *the AI decides, the engine measures*. Agents generate code; deterministic tools (tests, type checkers, mutation testing, continuous integration) decide whether it is acceptable; a human approves every merge. The method combines per-task branch isolation; task-scoped state files from which shared views are derived on demand and never committed; strict test-driven development, with the red step verified in continuous integration; a two-tier execution model, with single tests run locally and everything else in parallel cloud jobs; mutation testing restricted to modified files, in which only a human can accept a mutant as equivalent; a traffic light, task durations and CI usage derived by fixed rules from measured data rather than reported by agents; and enforcement layers that do not depend on the prompt. The reference implementation targets a Python backend; the stack-specific rules are meant to be adapted to other stacks. The rules were derived from failures the author observed in practice. This paper states the method, maps each failure mode to a mechanism and to its residual risk, lists the limitations, and proposes an evaluation protocol. It reports no performance measurements: whether the method improves outcomes remains to be measured.

Keywords: AI-assisted development; coding agents; multi-agent workflows; continuous integration; mutation testing; test-driven development; large language models.

1 Introduction

Coding agents such as Claude Code, Gemini-based tools and Cursor can read a repository, edit files, run commands and open pull requests. A developer can start several of them at once, each in its own terminal. The bottleneck then moves from writing code to keeping the work coherent and verifying it: who is working where, which state is current, and whether the code does what the agent says it does.

Advice for working with coding agents is mostly given as tips for prompting one agent. This paper addresses a different question: what repository-level rules, files and checks keep *several* agents from interfering with each other and make their output verifiable by tools that do not share the model's blind spots?

The paper's contributions are limited and explicit:

1. A catalogue of seven failure modes observed while working with several agents on one repository (Section 2).
2. The method itself: a set of rules, state files and checks (Section 4), whose full rule set is given in Appendix A.
3. A separation between what the method requires and what its reference implementation chooses, and the expected gains stated as measurable hypotheses (Sections 4.11 and 5).
4. A mapping from each failure mode to the mechanism that addresses it and to the risk that remains (Section 6).
5. An evaluation protocol that would let others test the method against a baseline (Section 8).

The reference implementation targets a Python backend, and part of the rule set names Python tools. The method itself does not depend on the language: the stack-specific rules are meant to be re-written for another stack (Section 4.10, and the stack note in Appendix A).

AFAW is a practitioner methodology. It was distilled from the author's experience and has not been validated in a controlled study. No claim about speed, cost or defect rates is made here.

2 Failure modes observed

The failure modes in Table 1 were observed by the author while working with coding agents. They were not logged systematically, so their frequency is unknown, and the list reflects one person's experience.

ID	Failure mode	What it looks like in practice
F1	Wrong-branch work	An agent starts editing files on whatever branch is checked out and collides with work that other agents are doing.
F2	Shared-state conflicts	Two agents update the same shared context file (a last-context or pending-tasks file) and produce merge conflicts.
F3	Unmeasured claims	An agent reports a result (coverage, a passing suite, a speed-up) that it never measured.
F4	Vacuous tests	Tests pass regardless of the behavior of the code they are meant to check.
F5	Heavy local runs	Full test suites launched on the developer's machine block the workflow.
F6	Unreviewed integration	Nothing prevents a direct push to the main branch or a merge without human approval.
F7	Unconstrained side effects	In systems that embed an LLM, a model output triggers a write with side effects without independent validation.

Table 1. Failure modes addressed by AFAW.

3 Design principles

P1. The AI decides, the engine measures. Every number (coverage, metrics, risk) comes from a deterministic function, never from an estimate, a rounding or an extrapolation by the model. The model proposes; tools measure.

P2. Determinism. The same input yields the same numbers. Two identical runs that differ are treated as a bug.

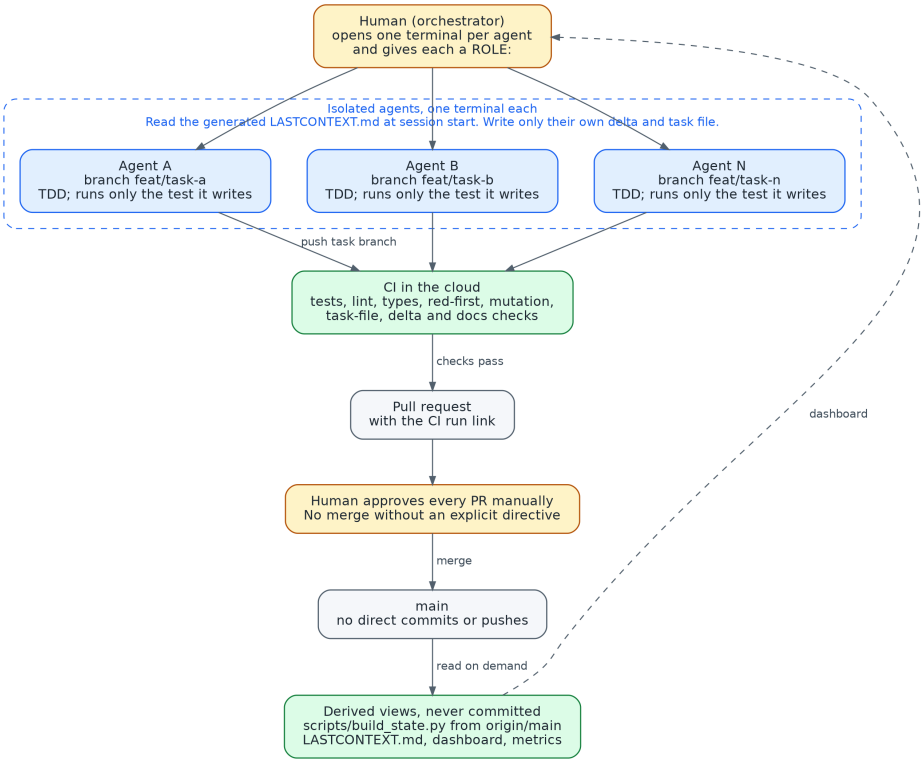


Figure 1. Overview of AFAW. The three agents are isolated, one terminal each. Blue: agent action; amber: human; green: CI or deterministic check; purple: state files; red: blocked.

P3. Parsimony. Use the simplest mechanism that solves the observed problem, and add a heavier one only with evidence that the simple one fails. Do not introduce opaque components (for example a trained classifier) that then have to be monitored.

P4. Human authority. Agents propose and structure changes. A human approves every pull request and decides every merge.

P5. State as code. Project state and agent context are versioned files in the repository, so they are diffable, reviewable and reproducible. Only source state is versioned: any view derived from it is computed when needed and never committed.

P6. Fail loudly, do not fabricate. If a check cannot run, it returns a real error, never an empty result that looks like a pass.

P7. Advice is not a guarantee. Instructions in a prompt can be ignored or misread. Rules that matter are also enforced by mechanisms outside the model.

4 The method

Figure 1 shows the whole flow. The human opens one isolated terminal per agent and gives each a role. Each agent works on its own branch, follows test-driven development, runs locally only the single tests it is writing, and pushes. Continuous integration (CI) runs every other check in the cloud, including a check that each new test failed before the change. The human approves the pull request. Shared views of the project state are derived from per-task files when needed and are never committed.

4.1 Roles and session start

In its first message, the agent asks the human for its role (`ROLE:`). A session-start hook builds the shared views from `origin/main` with `scripts/build_state.py` (Section 4.3) and injects the generated last-context file (Section 4.5) and the roadmap file (`PENDING.md`); if the build fails, the agent receives the error instead of a stale copy. The agent also reads the last-context file of its role. Before touching any file it runs `git status` and `git branch`, because other agents are working in the same repository. It never works on the branch that happens to be checked out: it creates a new, isolated branch first.

4.2 Task isolation and the life of a task

Each task gets its own branch, named after the task with a prefix (`feat/`, `fix/` or `chore/`) and a clear description. Work is committed in atomic units: one feature, one commit, in the Conventional Commits format [4]. The agent never commits or pushes to `main`. Figure 2 gives the twelve steps from the first question to the deletion of the branch.

At the end of a task the agent updates the affected documentation and its own task files, and records them in a separate documentation commit that contains documents only. A decision that constrains later work is recorded as an architecture decision record in the same pull request (Section 4.5). The agent then asks whether to create the pull request, unless the human has already asked for it. The pull request carries a title, a description and the link to the CI run. The human approves it manually, and merging requires an explicit human directive.

4.3 Isolated state and derived views

Agents never edit shared state. Each agent writes only two files: its delta (`context/tasks/task_NNN_context.json`) and its task file (`state/tasks/task_NNN.json`). Because no two tasks write the same file, two pull requests cannot conflict on state. The shared views are derived from these files:

- the pending-tasks view (`pending.json`);
- the global last-context file (`LASTCONTEXT.md`, Section 4.5);
- one last-context file per role.

They are produced by `scripts/build_state.py`, a pure function of the per-task files: the same inputs give byte-identical output (P2). The views are written to a build directory listed in `.gitignore`, and CI rejects a pull request that commits one. An agent builds them from `origin/main` at session start and whenever it needs a status report. There is no regeneration step after a merge, no bot pull request, and no copy of the state that can go stale on a working branch.

Two consequences follow. First, task identifiers must be unique across parallel branches, so CI rejects a pull request whose task identifier already exists on `origin/main`. Second, a view built from `origin/main` shows only merged tasks: an agent does not see work in flight on other branches. This is deliberate, because unmerged work has not been approved, but it means that two agents can start overlapping tasks. The roadmap in `PENDING.md`, where the human assigns work, is the guard against that.

A CI script (`scripts/check_task_delta.py`) checks the delta against the diff. It lists the files changed by the pull request with `git diff --name-only` against the merge base, keeps those under the code folders set in `state/config.json`, and requires that set to equal the `files_touched` field of the delta. A missing or extra entry fails the check. The field is there-

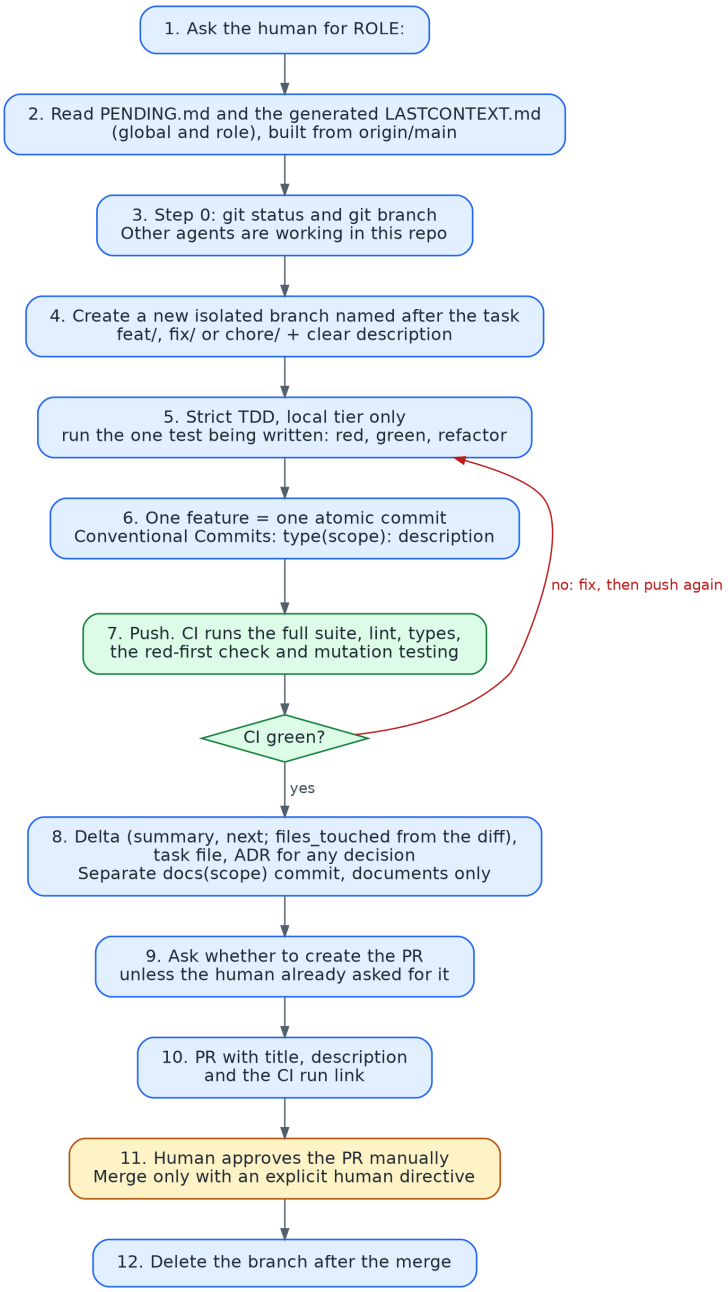


Figure 2. The life of a task, from asking for a role to deleting the branch after the merge.

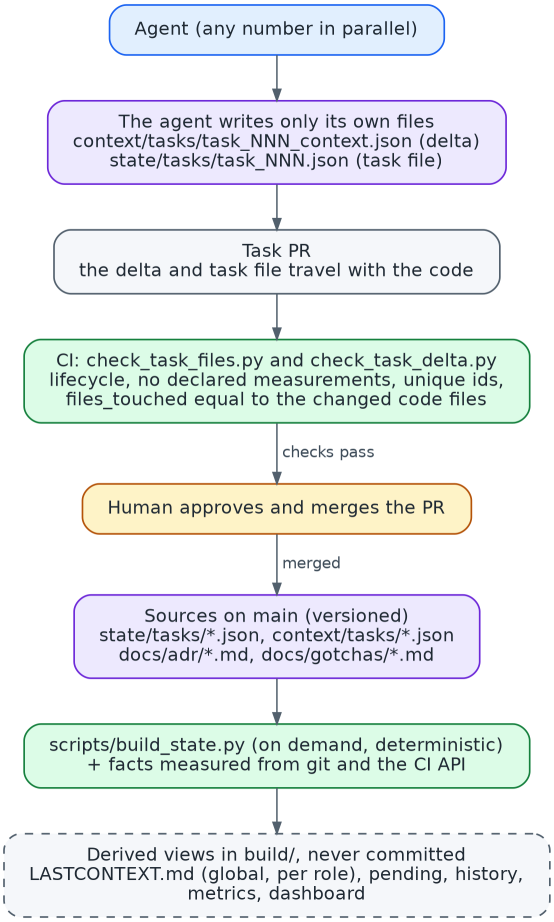


Figure 3. State and context. The agent writes only its own delta and task file; CI checks the delta against the diff; the shared views are derived from `origin/main` on demand and never committed.

fore a measured value, not a declaration by the agent. The agent fills it with the same script (`--write task_NNN`), so it never writes the list by hand. Figure 3 shows the flow.

A status report is built from `origin/main`, never from a working branch. What it contains, and how each value is obtained, is described next.

4.4 Task lifecycle, traffic light and timing

A task file holds only declared values: identifier, title, role, type, status, priority, difficulty, the tasks it depends on (`blocked_by`), its branch and its owner. The owner is an agent by default, or the human for actions only the human can take (approve a pull request, rotate a key, change a repository setting); such a task needs no branch, and a pull request may not change both the owner and the status of a task. The status is one of `backlog`, `pending`, `in_progress`, `done` and `cancelled`. A pull request sees only its two endpoints: a task that went from pending to in progress to done on its branch arrives as pending to done. So any open status may reach `done` in one pull request, a new task may arrive already done, and `done` and `cancelled` are final. A CI script (`scripts/check_task_files.py`) rejects a pull request that reopens a final status, deletes a task file, adds a task identifier that already exists on the target branch, or marks a task in progress or done on a branch other than the pull request's own. Task files are never deleted, so the files of done tasks are the history of the project. A task becomes done on `main` only through the merge of the pull request that marked it so.

Measured values are never declared. The same check rejects a task file that contains a timestamp, a CI result or a traffic light. They are collected by `scripts/collect_facts.py` from git and from the pull-request and CI interfaces of the hosting platform (Table 2). Pull-request timestamps are used instead of branch commit dates because a squash merge or a rebase rewrites the commits of the branch.

Fact	Definition	Source
`created_at`	Commit that added the task file to `main`	git
`started_at`	Earliest commit of the task's pull request	Pull-request commits
`pr_opened_at`	Creation of the pull request	Pull-request API
`first_green_at`	Earliest time at which every CI run of one commit had finished green	CI API
`merged_at`	Merge of the pull request	Pull-request API
`last_activity_at`	Latest commit or CI run on the branch	git, CI API
CI status	State of the runs of the latest commit: success, failure or pending	CI API
CI usage	Runs, failed runs, jobs, queue time, job minutes	CI API

Table 2. Measured facts per task. None of them is written by an agent.

Traffic light. The light is a rule, not an assessment by the agent (P1). The conditions are checked in this order:

- *red*: a task it depends on is not done, or CI fails on the latest commit of its branch;
- *no data*: a task in progress for which no fact was measured, shown as such rather than as green (P6);
- *yellow*: a task in progress with no commit and no CI run for more than a set number of days (`stale_days` in `state/config.json`);
- *green*: every other open task.

Backlog, done and cancelled tasks have no light. The priority is set by the human. The difficulty is a declared estimate and is labelled as such in every view.

Durations. Three wall-clock durations are derived for each task, and kept apart because they measure different things:

- *agent time*, from `started_at` to `first_green_at`: work until CI first passes;
- *review latency*, from `pr_opened_at` to `merged_at`: the human's time, not the agent's;
- *lead time*, from `started_at` to `merged_at`: the total.

CI usage. For each task, the number of runs and of failed runs, the queue time (the sum over jobs of the start time minus the creation time) and the job minutes (the sum over jobs of the duration rounded up to a whole minute, the unit in which GitHub bills standard runners). Price multipliers for other runner types are not applied; the platform's billing report remains the ground truth for cost.

Views and dashboard. `scripts/build_state.py` derives the pending view, a history of done tasks ordered by merge time, the last-context views, a metrics file and a static HTML dashboard. Aggregates are the median and the quartiles over done tasks, with the number of tasks for which a value is missing; a missing value is counted, never filled in. Progress is the number of done tasks over the number of tasks not cancelled. It is a count: every task weighs the same, because the only size available, the difficulty, is an estimate. The dashboard also shows the tasks by status and the tasks merged per day, as charts drawn when the page is

built.

The reference time is an explicit input: by default the commit time of the ref being read, never the clock, so the same ref, facts and reference time give byte-identical views (P2). The dashboard is rebuilt on every push to `main`, and also on a schedule: the light of a task in progress depends on the CI and the activity of its branch, which change without any commit to `main`. A scheduled run passes its own time as the reference, so inactivity is measured against the run. The dashboard contains no scripts and loads nothing when viewed. It can be opened locally, kept as a CI artifact, or published as a static site; a public site exposes the backlog.

4.5 Project knowledge and the last-context file

State files say *what* is happening; they cannot say *why*. A decision recorded as one line of data does not say which problem it solves or which alternatives were rejected, and a later agent or person who does not know the reason is likely to reverse it. Recording the same knowledge twice, once as data and once as prose, creates a second problem: the two copies drift apart. AFAW therefore keeps one source per kind of knowledge, and the source of knowledge is prose.

- **Decisions** are architecture decision records [8] in `docs/adr/NNNN-slug.md`, with the sections Context, Decision, Alternatives considered and Consequences. Rule: a decision that constrains later work is recorded as an ADR in the pull request that introduces it, and the human accepts it there.
- **Gotchas**, operational traps that still apply, are short records in `docs/gotchas/NNNN-slug.md`, with the sections Symptom and What to do.
- **Architecture and environment** descriptions are ordinary documents, written and updated by hand.
- **What was done and what was not** in a task is in its pull request and in the `summary` and `next` fields of its delta.

Each record starts with a small front-matter block that holds only what a program needs: status, date, tags and links to other records (`supersedes` for an ADR; `resolved_by` or `promoted_to` for a gotcha). A CI script (`scripts/check_docs.py`) checks file names, front matter, required sections, the consistency of the `supersedes` chain, and that relative links in the documentation resolve. It cannot tell whether a decision that should have been recorded is missing; that remains the reviewer's judgement.

The global last-context file, `LASTCONTEXT.md`, is generated by `scripts/build_state.py` and never written by hand. Its sections follow the questions an agent asks at the start of a session: where things stand (progress, and each task in progress with its light and the summary and next steps of its delta); what is waiting on the human; the next tasks in order; the decisions in force and those proposed; the active gotchas; and the reference documents to read. Decisions and gotchas appear as one line each, with the path to the record, so the file is an index to documents rather than a copy of them. A per-role file shows the same index filtered by tags configured for the role, together with the tasks most recently merged in that role.

Nothing in the file grows by accumulation. The parts about tasks are bounded by the open tasks. A record leaves the index only for an explicit reason, never because of its age: an ADR is superseded or deprecated; a gotcha is resolved, with the task that fixed its cause, or *promoted*, with the mechanism that now enforces it (a hook, a script, a configuration default or a test). Promotion is the preferred exit, because a rule enforced by a mechanism no longer depends on the agent reading it (P7). As a last control, the number of active records is measured against a configured budget; above it, the dashboard reports that a review is due, and the human prunes in a pull request. No record is deleted or summarized automatically: deciding that knowledge

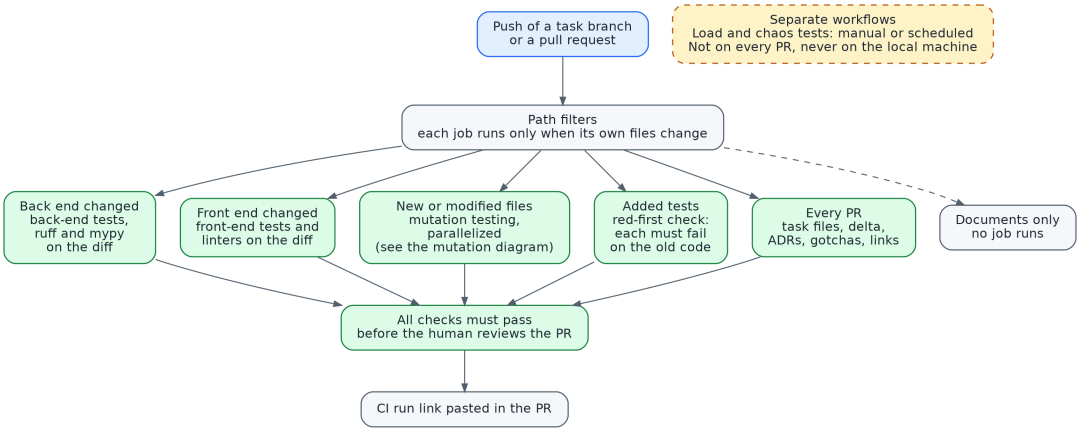


Figure 4. The CI pipeline. Path filters route a change to parallel jobs, including the red-first check; load and chaos tests are separate workflows.

no longer applies is a judgement, and a generated summary would be an unmeasured claim (P1).

4.6 Verification: local and CI tiers

Development is test-driven in the strict sense: the agent writes a test, confirms that it fails, writes the minimum code that makes it pass, and refactors with the test green. For a bug, the first step is a test that reproduces it and fails. When the task is to add tests to existing code, only the test folder is edited. If a new test fails against the original code, the test is presumed wrong unless there is evidence of a real defect, in which case it is marked as an expected failure with the reason recorded, and the source is not changed to make it pass.

The red–green loop needs a test run at every step, and a push to CI per step would be too slow to follow. Execution therefore has two tiers. On the local machine, the agent runs only the tests it is writing, one test or one test file at a time. This tier must finish within seconds and must not need external services. Everything else runs in CI.

A red step seen locally is still a claim by the agent (F3). CI turns it into a measurement with a red-first check (`scripts/red_check.py`). The check takes the test functions added by the pull request, runs them against the code at the merge base, and requires each of them to fail. A new test that passes on the old code did not drive the change, and the check fails. Some tasks add tests that are meant to pass on the old code: tests for existing code, and refactors with no change in behavior. Their task file declares the type (`tests-only` or `refactor`); the check is then skipped, and the declaration is shown to the reviewer. Within a feature, a single test that documents behavior the code already has is declared with a `characterization` marker and a reason, and is printed in the same way. A test that does not appear in the report of the old run did not run, and the check fails rather than count it as passing. On the pull request that introduced these scripts, the check rejected four tests; one of them passed on the old code for the wrong reason (it asserted only a non-zero exit, which a missing module also gives) and was rewritten.

The full test suite, linters, type checks, mutation testing and builds run in CI. Independent jobs run in parallel, and path filters make each job run only when its own files change: back-end changes launch back-end jobs, front-end changes launch front-end jobs, and document-only changes launch nothing. Linters run only on the modified files. Load and chaos tests run as manual or scheduled workflows, not on every pull request. Figure 4 summarizes the pipeline.

Verification rules apply to the checks themselves. A script that reports PASS does not prove that it measured the right thing, so internal consistency (two runs agree) is combined with a known external value. Credentials and configuration are validated before minutes of compute are spent.

4.7 Mutation testing on the diff

Mutation testing [5, 6] alters the source in small ways (mutants) and re-runs the tests. If the tests still pass, no test detected that change. The line may well be executed; no assertion depends on it. With N generated mutants, K killed and E equivalent to the original, the mutation score is $K/(N-E)$. Coverage and mutation score measure different things and should be reported separately.

AFAW applies mutation testing only to new or modified files, in parallel in CI (Figure 5). Before mutating, the baseline is validated: the suite must pass on unmodified code, otherwise every mutant would count as killed and a false 100% would result. Surviving mutants are fixed by writing or improving tests, and the run is repeated. A survivor believed to be equivalent follows the procedure below; it is never covered with an artificial test. The step is mandatory in critical modules (money, security, concurrency, public contracts), where a file below the minimum score blocks the pull request. The critical paths and the minimum score are defined per project in `state/config.json` (`mutation_critical_paths` and `mutation_min_score`).

The denominator is where the gate can be gamed. Whether a mutant is equivalent is undecidable in general, and an agent that marks survivors as equivalent lowers $N-E$ until the gate passes: an unmeasured claim again (F3). AFAW therefore separates the proposal from the decision. The agent may propose a survivor as equivalent, with a justification, in the pull request. Only the human records an accepted equivalence, in `state/equivalent_mutants.json`. A hook denies agent writes to that file, and CI lists every change to it in the pull request summary. Each entry names the file, the mutant and a hash of the file content, so the entry expires when the file changes. The gate uses $K/(N-E_h)$, where E_h counts only equivalences accepted by the human. The raw score K/N is reported next to it, and expired equivalences are listed rather than counted. The mutation engine stays project-specific; `scripts/mutation_gate.py` applies the gate to its results.

4.8 Human authority and enforcement layers

The rules “never commit or push to `main`”, “never merge without an explicit human directive” and “the human approves every pull request” are written in the agent instruction file. Because that file is advice (P7), they are also placed under mechanisms that do not depend on the model (Figure 6). Hooks intercept commands before they run. Hard-deny rules let `PENDING.md` be edited but never deleted or emptied, deny agent writes to `state/equivalent_mutants.json` and to the generated files under `build/`, and deny a push to `main`. CI checks block a pull request that fails tests, lint, types, mutation, the red-first check, the task-delta check or the task-file check. Branch protection on `main` makes the server reject a direct push. The human approval is the last step.

Hooks are a best-effort filter, not a security boundary: a command can be phrased in a way a pattern misses. Branch protection is the barrier that does not depend on the agent.

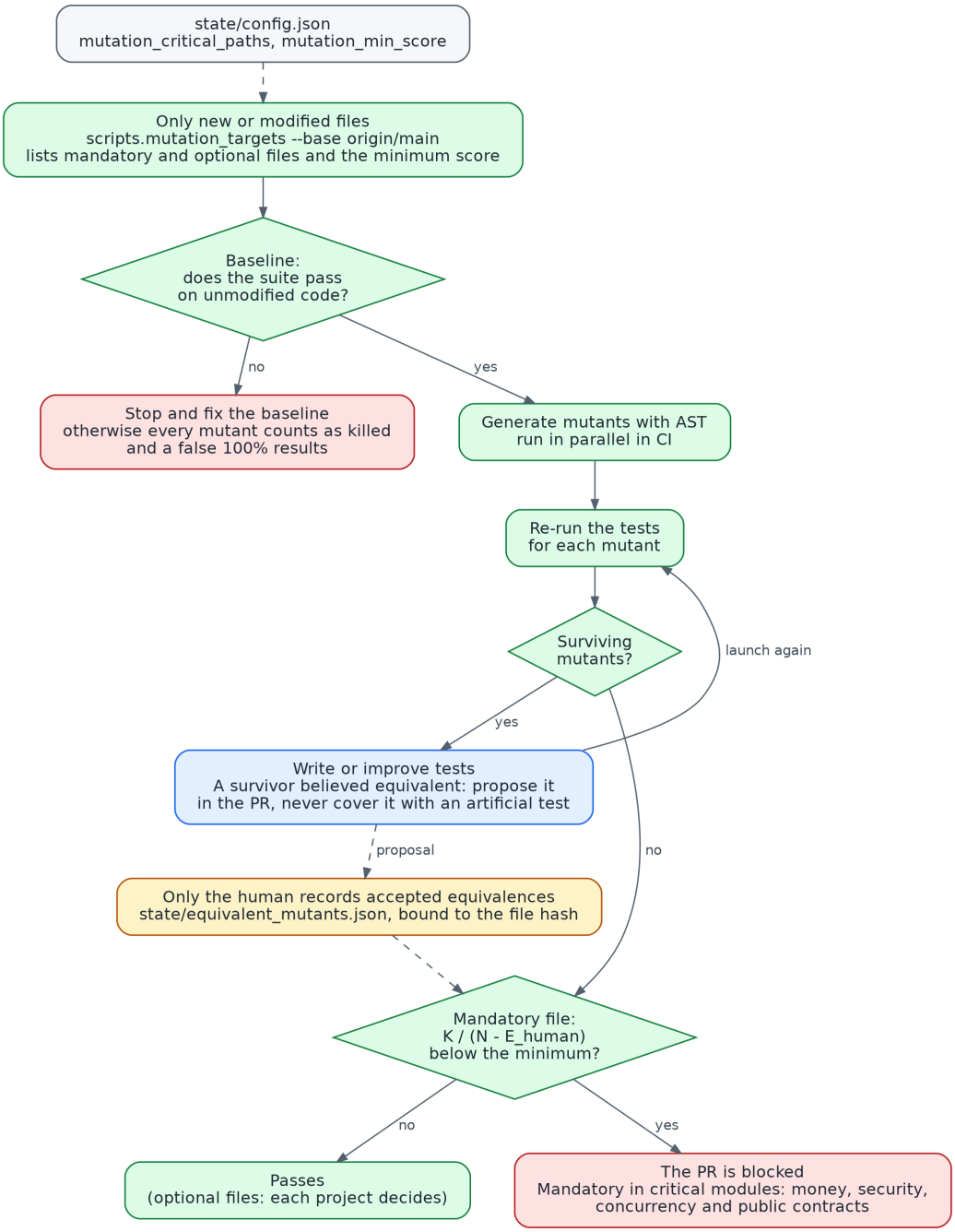


Figure 5. Mutation testing on the diff, with the baseline check, human-only acceptance of equivalent mutants, and the blocking rule for mandatory files.

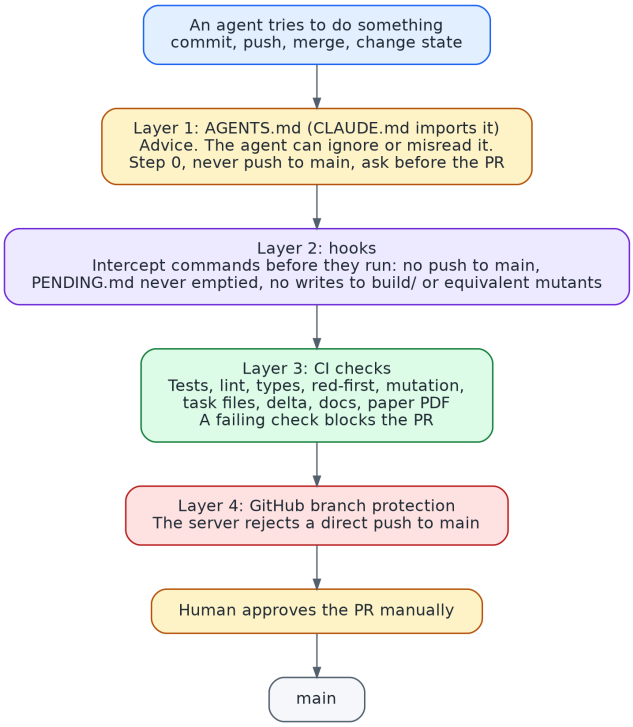


Figure 6. Control layers. The layers are shown in a rough order; in practice each acts at a different moment.

4.9 A module for systems that embed an LLM

When the product itself contains LLM agents (F7), AFAW adds rules that follow the same principles. The LLM never executes tools with side effects: only deterministic code does, and every write tool is idempotent by a request identifier. LLM output is never trusted: it is revalidated on the server with strict schemas and business limits. The user-facing agent has minimum privilege, and a prompt-injection filter runs before a message reaches it. Every component is declared fail-open or fail-closed; guards and judges fail closed, while context retrieval and tracing fail open. Each tool call is authenticated and authorized on the server, and every invocation, including denied ones, is written to an audit log with personal data masked. Model judges are evaluated on the production code, and false approvals are reported first because they are the dangerous error.

4.10 Adapting to other stacks

Most rules are stack-independent. The rules on code standards, architecture, concurrency and messaging, debugging, and infrastructure (Appendix A, sections 4–10 and 14–16) assume a Python backend with asynchronous I/O, a relational database, a message broker, Terraform and the Model Context Protocol. For other stacks they should be adapted or removed.

4.11 Method and reference implementation

AFAW is the method; the code in the repository is one way to implement it. Tests can be written in any framework, mutation testing can use any engine or be left out outside critical modules, and CI can be any system that runs checks outside the agent's machine. Some parts cannot

be removed without bringing a failure mode back. If per-task state files are replaced by one shared context file, agents working at the same time overwrite each other again (F2). Table 3 separates the three kinds of part.

Core: removing it brings a failure mode back	**Replaceable: same intent, another tool**	**Optional**
State written per task, never shared; shared views derived (F2)	The CI system and the test framework	The dashboard and its charts
Measured values, never declared by the agent (F3)	The mutation engine	Mutation testing outside critical modules
Heavy checks outside the agent's machine (F5)	Hooks, for another agent or tool	Timings and CI cost
One branch per task; no push to `main`; human sign-off (F1, F6)	The format of decision records	The per-role last-context files
	The retrieval stack of the assistant	An assistant that answers from the records (Section 5.1)
A red step verified by a tool, not reported by the agent (F4)	The language of the rules file	

Table 3. What the method requires and what the reference implementation chooses.

5 Why it pays off

The mechanisms have a cost: a one-time setup, and per task a task file, a delta and sometimes a decision record. This section states, in plain terms, what the method is expected to give back. Each item is a hypothesis with a metric in Section 8.

It is cheaper to run. An agent that runs the full test suite on a slow machine sees failures caused by the machine, not by the code, and retries. Every retry sends its whole context again, and the loop can last an hour. With two tiers, the agent runs only the test it is writing, and CI runs the rest in parallel on clean machines and returns one verdict. A minute of CI costs much less than an hour of an agent looping, so the method is expected to cost less than it adds (RQ3).

It works for one person. A single developer who leaves agents running has no reviewer. The checks, the branch protection and the hooks do the reviewing that nobody else is there to do. Small projects are where agents are most often left alone, and where these guards are most needed.

It makes agents' work visible to a team. With two or ten developers, each running their own agents, the problem becomes knowing what the others' agents changed, why, and where each task stands. Every task, decision and status is recorded in the same place and the same format, whoever or whatever did the work, and the board and the last-context file show it to everyone. The setup is done once, from the template.

It documents the project as it goes. Every task leaves a summary, the next steps and, for each decision, a record of why it was taken. Without this, when agents do the work, nobody knows afterwards what was done or why. The generated last-context file is an index to that record.

A new agent understands the project faster. An agent that starts from nothing has to read the whole repository to work out what was decided and why. With the index, it reads one file

and follows links to the decisions it needs. That is less context, fewer tokens and less time, and the saving grows with the project (RQ5).

Fewer status meetings. Because what was done, why, and where each task stands is recorded when the work happens, a person does not have to gather others to find out. The board answers most of it, and an optional assistant can answer the rest in conversation (Section 5.1, RQ6).

5.1 Optional: an assistant that answers from the project's records

The records the method produces can feed an assistant that answers questions about the project by retrieval-augmented generation (RAG): it searches the documents and answers from what it finds. The assistant is not part of the method. It is an optional use of what the method records, and its benefit is a hypothesis like the others.

What it can replace. Status meetings. The questions they exist for (what was done, where each task stands, what is blocked and why, why something was decided) are answered by the board, the last-context file and the decision records. The assistant answers the same questions at any time, to anyone, without taking an hour from several people.

Why the records suit retrieval. Each document covers one subject: one decision record per decision, one gotcha per trap, one delta per task. Documents split into self-contained passages without cutting an idea in half. The front matter carries status and tags, so the assistant can keep to decisions in force and skip those that were superseded; retrieval over loose documents mixes old decisions with current ones. The last-context file is a ready-made entry point, and every fact has a path, so each answer can cite its source and anyone can check it.

What it cannot replace.

- *Meetings where things are decided.* Priorities, trade-offs and disagreements need people. The records say what was decided, not what to decide, although such meetings get shorter when nobody has to reconstruct the current state first.
- *What was never written down.* The assistant knows only what is recorded. A decision taken in conversation and never turned into a record does not exist for it; the method pushes to record decisions but cannot force it.
- *Records that went stale.* A decision record whose text stopped being true, and was never superseded, produces a confident wrong answer.

Two rules, both following from the design principles.

- *Numbers are copied, never paraphrased.* Progress, lead time, CI cost and every other figure are quoted from the derived views. An assistant that summarizes metrics in its own words can produce numbers nobody measured (P1).
- *The assistant is read-only.* It answers questions; it never writes to the repository, opens pull requests or changes state (P4).

6 From failure mode to mechanism

Table 4 links each failure mode to the mechanism that addresses it and states the risk that remains. The last column matters as much as the second: none of the mechanisms removes its failure mode.

ID	Failure	Mechanism	Residual risk
F1	Wrong-branch work	Step 0 (`git status`, `git branch`); one branch per task; one isolated terminal per agent.	Step 0 detects a shared working directory but does not prevent it. Agents sharing one working tree can still overwrite each other's uncommitted files. The rule itself is only advice.
F2	Shared-state conflicts	Task-scoped files that no two tasks share; shared views derived on demand from `origin/main` and never committed; unique task identifiers checked in CI.	Agents do not see unmerged work on other branches and may start overlapping tasks; assignment in `PENDING.md` is the only guard.
F3	Unmeasured claims	Rule "never claim a result that was not measured"; CI as the gate; red-first check; `files_touched` compared with the diff; timestamps, CI results and traffic lights rejected in task files and derived from measured facts; run link in the pull request; fail loudly.	A person must still follow the link; a compliance failure by the agent is caught only if someone checks.
F4	Vacuous tests	Test first, with the failure verified in CI by the red-first check; mutation testing with baseline validation; equivalent mutants accepted only by the human.	Mutation score is not correctness. Tests and code written by the same model can share blind spots. Tasks declared `tests-only` or `refactor` skip the red-first check.
F5	Heavy local runs	Two tiers: single tests locally; everything heavy in CI, in parallel, filtered by path.	Queue and start-up latency in the CI tier; quota of CI minutes on private repositories; the time limit of the local tier is a rule an agent can exceed.
F6	Unreviewed integration	Rules, hooks, CI checks, branch protection, human approval.	Hook patterns are best-effort. A solo developer cannot require an approving review from a second person.
F7	Unconstrained side effects	Deterministic execution layer, strict revalidation, least privilege, audit log.	Schema and validation bugs; attack surfaces beyond the ones anticipated.

Table 4. Failure modes, mechanisms and residual risks.

7 Limitations and threats to validity

- **No measurement.** This paper reports no data on speed, cost, tokens or defects. The benefits of the method are hypotheses.
- **Single origin.** The rules come from one practitioner's experience with a small set of projects. The failure modes may not represent other teams or codebases (selection bias).
- **Prompt compliance.** An instruction file does not guarantee that a model follows it. How often the rules are violated, and how often the mechanisms in Section 4.8 catch a violation, has not been recorded.
- **Mutation testing.** Mutation score measures how sensitive the tests are to changes in the code. It does not show that the code meets its requirements, and equivalent mutants cannot be killed by any test. Whether a survivor is equivalent is a human judgement and can be wrong. The AST-based operators are limited to what the engine implements.
- **Red-first check.** The check shows that a new test fails on the old code, not that it fails for the right reason. A test that fails because of an import error or a typo passes the check. A characterization marker is a declaration the reviewer must judge.
- **Shared blind spots.** If the same model writes the code and the tests, deterministic checks can pass on a wrong interpretation of the requirement. Independent tests or a second source of ground truth are needed to detect this.
- **Two-tier execution.** Allowing single tests locally and sending everything else to CI is a

judgement about where to draw the line. Neither the local time limit nor the alternatives (all checks local, all checks in CI) have been compared.

- **Wall-clock timing.** The durations include queue time, waits for the human and periods when nobody worked on the task; they are not effort. `started_at` comes from commit dates, which are set by the clock of the machine that made the commit and can be changed; pull-request and CI timestamps come from the server. The CI facts are read from GitHub's interfaces in the reference implementation.
- **Traffic-light thresholds.** The rules are fixed, but the threshold `stale_days` is a choice, and a green light says only that no rule fired.
- **Progress by count.** Progress treats every task as the same size and falls when tasks are added. It measures how much of the recorded backlog is done, not how much of the work.
- **Unrecorded decisions.** The checks verify the records that exist, not the decisions that were never recorded. The knowledge budget is a threshold chosen by hand, and a record can remain active after it stops being true until someone closes it.
- **Visibility of unmerged work.** Views built from `origin/main` hide tasks in flight. How often this leads to duplicated or conflicting work has not been measured.
- **Human review remains necessary.** The method reduces how much code a person must read, but intent, design, security and the tests themselves still need human judgement.
- **Stack specificity.** Part of the rule set assumes a Python backend (Section 4).
- **Stale knowledge.** Documentation that is no longer true misleads more than no documentation. The derived views cannot go stale, because they are rebuilt, and a decision record can be superseded. Nothing detects a record whose text stopped being true while nobody replaced it; that is left to the human and to the knowledge budget.
- **Model drift.** Agent behavior depends on the model and tool version. Any measurement should record both and the date.
- **On the name.** "Anti-fragile" follows Taleb's term for systems that gain from disorder [9]. Here it is used informally: failures caught by deterministic checks are turned into stronger tests and rules. This paper does not show that the method gains from stress in Taleb's formal sense.

8 Proposed evaluation protocol

The claim that AFAW helps should be tested, not assumed. This section proposes a design that others (or the author) can run.

Research questions.

RQ1 Does state isolation with derived, uncommitted views reduce merge conflicts and wrong-branch commits compared with agents sharing context files?

RQ2 Do the red-first check and the mutation gate reduce escaped defects and vacuous tests?

RQ3 What does the method cost in elapsed time, tokens, CI runs, CI queue time and CI minutes?

RQ4 How much human review effort does each pull request need, and what does the reviewer find?

RQ5 How much faster, and at what token cost, does a new agent answer questions about the project when it starts from the generated last-context file and the decision records, compared with reading the repository from nothing?

RQ6 With an assistant that answers from the project's records, how much status-meeting time

do people spend, and how often are the assistant's answers correct and backed by a cited source?

Design. Run the same task set under matched conditions: (A) a baseline with minimal agent instructions; (B) full AFAW; and ablations of B that remove one component at a time (state isolation, the red-first check, the mutation gate, two-tier execution). Fix and record the model, the tool version and the date. Randomize the order of tasks and repeat runs, because agent output is not deterministic. Tasks can come from the project's own backlog. Public issue-resolution benchmarks with hidden tests, such as SWE-bench [7], can supply an independent ground truth that the agents' own tests cannot.

Metrics. Table 5 lists the measures and where each can be read from.

Metric	Definition	Source
Merge conflicts per PR	Conflicts that need manual resolution	Git history
Wrong-branch commits	Commits made outside the task branch, or blocked attempts	Hook and server logs
Agent time per merged task	`started_at` to `first_green_at` (Section 4.4)	Pull-request commits, CI API
Review latency per PR	`pr_opened_at` to `merged_at`	Pull-request API
Lead time per merged task	`started_at` to `merged_at`	Pull-request API
Tokens per merged task	Model tokens spent until merge	Agent usage logs
CI runs per merged task	Runs, and runs that failed	CI API
CI queue time per merged task	Sum over jobs of start minus creation	CI API
CI minutes per merged task	Job minutes rounded up per job, reconciled with the minutes billed	CI API, CI billing data
Escaped defects	Failures found by hidden tests or after merge	Hidden tests, issue tracker
Mutation score and coverage	Reported separately for merged files; gated score, raw score and proposed equivalences side by side	Mutation engine, coverage tool
Red-first failures	Added tests that passed on the old code	CI logs
Human review minutes per PR	Time a reviewer actively spends before approving (effort, unlike review latency)	Reviewer log
Rule violations	Times an agent broke a rule, and by which mechanism it was caught	Hook, CI and protection logs
Onboarding cost	Tokens, time and correct answers for a fresh agent on a fixed set of questions about the project, with and without the knowledge files	Agent usage logs, answer key
Status meetings and answer quality	Minutes of status meetings per week; share of the assistant's answers that are correct and cite a source	Calendars, answer key

Table 5. Proposed metrics.

Reporting. Publish the raw data and the exact instruction files. Report intervals and not only means. Report false approvals first and negative results as well. State the model, tool version and dates so that the result can be read against later models.

9 Related work

AFAW builds on established practices rather than replacing them. Test-driven development [2] provides the red–green–refactor loop. Mutation testing goes back to DeMillo, Lipton and Sayward [5], and Jia and Harman [6] survey its development. Short-lived branches from an always-deployable main branch follow GitHub Flow [3], and commit messages follow the Conventional Commits specification [4]. Decision records follow Nygard's architecture decision records [8]. Instruction files for coding agents are now a common convention; the AGENTS.md format [1] is one example, and `CLAUDE.md` is another. The term "anti-fragile" is Taleb's [9]. SWE-bench [7] evaluates language models on real repository issues with hidden tests and is relevant to the evaluation protocol.

This paper does not survey the fast-moving literature on coding agents. Related work is limited to the sources on which the method directly depends.

10 Conclusion

AFAW is a set of rules, state files and checks for working with several coding agents on one repository. Its central choice is to let models generate and let deterministic tools verify, with a human approving every merge. It targets seven observed failure modes and states the risk that remains for each. Its design can be judged now: what it requires, what it enforces and what it leaves unsolved are stated. Its benefits cannot yet: it is a hypothesis about how to work, not a demonstrated result. The expected gains (Section 5) are specific and measurable, and the reference implementation already records the timings and CI costs they need. The protocol in Section 8 is offered so that the hypothesis can be tested, and refuted if the data say so.

Declarations

Availability. The reference boilerplate (agent instruction file, state layout, scripts and diagrams) is distributed as a repository template, `https://github.com/agustindiazcano/afaw-anti-fragile-agentic-workflow`, which also holds this paper (`docs/paper/afaw.pdf`) and its source; its licence is stated in the repository. Official DOI: 10.5281/zenodo.23050310.

Use of AI assistance. An AI assistant (Claude, Anthropic) was used to help draft and edit this paper and to produce the diagrams. The author defined the method, reviewed the text and is responsible for its content.

References

[1] AGENTS.md. *A simple, open format for guiding coding agents.* `https://agents.md/` (accessed 29 September 2026).

[2] K. Beck. *Test-Driven Development: By Example.* Addison-Wesley, 2002.

[3] S. Chacon. GitHub Flow. Blog post, 31 August 2011. `https://scottchacon.com/2011/08/31/github-flow/`.

[4] Conventional Commits. *Conventional Commits Specification, version 1.0.0.* `https://www.conventionalcommits.org/en/v1.0.0/`.

[5] R. A. DeMillo, R. J. Lipton and F. G. Sayward. Hints on test data selection: help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi:10.1109/C-M.1978.218136.

[6] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi:10.1109/TSE.2010.62.

[7] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[8] M. Nygard. Documenting architecture decisions. Blog post, 15 November 2011. `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions`.

[9] N. N. Taleb. *Antifragile: Things That Gain from Disorder.* Random House, 2012.

A Complete rule set

This appendix reproduces the complete instruction file that the agents receive, `AGENTS.md` (`CLAUDE.md` imports it). It is the source of truth for the method; the body of the paper summarizes it. The appendix is generated from the file by `scripts/rules_to_tex.py`, so the two cannot diverge. Two edits are made for typesetting: the traffic-light emoji of section 3 are written as the words green, yellow and red, and the file's opening heading is omitted.

Stack note. The reference implementation of these rules targets a Python backend (asynchronous I/O, `pytest`, `mypy`, a relational database, a message broker, Terraform and the Model Context Protocol). The stack-independent core is sections 0–3, 11–13 and 17–20. Sections 4–10, 14–16 name concrete Python tools and infrastructure, and should be re-written for another stack while keeping the intent of each rule.

> **This repository is `afaw-engine-gitlab` (AFAW Hands-off) and it lives on GitLab.** Work against the `gitlab` remote and `gitlab/main`; `origin` can be a stale GitHub copy.

Instructions for coding agents

Rules reusable in any project. `AGENTS.md` is the single source; `CLAUDE.md` imports it.

0. Language: always English

> **ALWAYS RESPOND IN ENGLISH.** Every reply, explanation, commit message, PR title/description and code comment is written in English, regardless of the language the user writes in.

The whole project is in English: source code, identifiers, comments, docstrings, documentation (README, ADRs, gotchas, white paper), state and context files, CI job names and log messages, Duo agent prompts and flow definitions, the web application and its API responses, and every commit, merge request and issue. A file in another language is a defect, whatever language the human uses in conversation.

1. Session start, tasks and context

1. **In the first message, always ask the human for `ROLE`:** (which role this agent has in the project).

2. **An agent always reads `LASTCONTEXT.md` and `PENDING.md` before anything else, with or without the hook.** At session start the hook builds the derived views from `origin/main` and injects both. Without the hook, run `git fetch` and `python -m scripts.build_state --ref origin/main`, then read, in order: `README.md`, the reference documents listed in `LASTCONTEXT.md`, `build/state/context/global/LASTCONTEXT.md`, `build/state/context/roles/<role>/LASTCONTEXT.md`, and the ADRs and gotchas the task needs.
3. **An agent writes only two state files:** its task file `state/tasks/task_NNN.json` (declared values: id, title, role, type, status, owner, priority, difficulty, blocked_by, branch) and its delta `context/tasks/task_NNN_context.json` (`summary`, `next`, `files_touched`). Timestamps, CI results and the traffic light are measured, never declared; CI rejects a task file that contains them.
4. **Status lifecycle:** `backlog`, `pending`, `in_progress`, `done`, `cancelled`. Any open status may reach `done` in one pull request; `done` and `cancelled` are final. Task files are never deleted: set `cancelled`. A new task takes the next free id; CI rejects an id that already exists on `origin/main`.
5. **Owner:** `agent` by default; `human` for actions only the human can take (approve a PR, rotate a key, change a repository setting). A human task needs no branch. A pull request never changes both the owner and the status of a task.
6. **When finishing a task:** write `summary` (what was done) and `next` (what is left) in the delta; fill `files_touched` with `python -m scripts.check_task_delta --base origin/main --write task_NNN`, never by hand; update the affected documentation; record any decision as an ADR (section 20). Commit these documents in a separate `docs(scope): ...` commit, documents only, so it does not launch tests (section 11).
7. **Generated files are never edited:** everything under `build/` (the `LASTCONTEXT.md` files, pending, history, metrics, the dashboard) is rebuilt by `scripts/build_state.py` and never committed.

2. Role and way of working

1. Act as a senior engineer and QA specialist: SOLID, Clean Architecture, small, modular, typed, testable, production-ready code.
2. **Never claim a result that was not measured.**
3. **Strict TDD (Red → Green → Refactor):** write the test first and confirm it fails; then the minimum code to make it pass; refactor with the test green.
4. Never present a function or endpoint as finished without its test.
5. For a bug, first write a test that reproduces it (it must fail) and only then fix it.
6. Tests for detectors or estimators use synthetic sequences with known noise and a known change point.
7. **Parsimony:** implement the simplest model or detector that solves the problem; use a heavier one only with evidence that the simple one fails.
8. Do not add opaque components (e.g. a trained classifier) that then have to be monitored.
9. Consult the current documentation of a library before implementing with it; do not do it from memory.

3. Project status report

When the project status is requested, build the views from `origin/main` (`git fetch`, then `python -m scripts.build_state --ref origin/main`) and answer from `build/state/pending.json`, never from the working branch or by reading `state/tasks/` one by one. Answer with a table of open tasks:

Task	Status	Light	Difficulty (1-5, estimate)	Priority (1-5)
Example: migrate the worker to async	in_progress	yellow no activity for 4 days	3	1

1. **The light is derived by rules, never judged by the agent:** red a blocker is not done, or CI fails on the latest commit of the task's branch · yellow in progress with no commit or CI run for more than `stale_days` · green otherwise · no data when nothing was measured. Copy the light and its reason from the view.
2. **Priority:** 1 = most urgent, 5 = least urgent. Sort by ascending priority.
3. **Difficulty** is an estimate and is labelled as one.
4. One row per task; no paragraphs inside the table. For a red task, give on a separate line the blocker and what is needed to unblock it.
5. Only real tasks; do not invent tasks to fill the table.

4. Code standards

1. Type hints on every public function, with a short docstring (what it measures/does and which file it writes).
2. `mypy --strict`; no `Any` unless justified in an inline comment.
3. Every function that performs I/O is `async`, and there is never blocking I/O inside `async` (use `asyncio.sleep`, `httpx.AsyncClient`, async drivers).
4. Every blocking subprocess call has a timeout; long-running processes are terminated or awaited; temporary working directories are deleted.
5. Configuration from environment variables with `pydantic-settings`; never hardcode secrets or connection strings.
6. Structured logging (`structlog`); never `print()`.
7. PEP 8 with a maximum line length of 100 characters.
8. No `from module import *`.
9. Do not use a generic `except Exception` without logging or re-raising the specific error.
10. No `TODO` comments: implement it or ask.
11. Do not add new dependencies without justifying them; prefer the standard library.
12. Single-purpose functions; one file, one responsibility; split a file before it exceeds ~600 lines.
13. Every module has its own test file (`pytest` + `pytest-asyncio`).
14. Tests run against their own database whose name ends in `_test`; the suite refuses to start otherwise.
15. All code lives in `src/` with absolute imports, so it works the same locally, in tests and in Docker.
16. Sample or demo data is loaded with separate scripts or jobs, never inside a migration.

5. Architecture

1. **The AI decides, the engine measures:** every number (coverage, metrics, risk) comes from a deterministic function, never from an estimate, rounding or extrapolation. Under the merge policy of section 18, the human approves the policy and the deterministic engine approves each merge.
2. **Determinism:** same input, same numbers. Two identical runs that differ are a bug.
3. **Layer separation:** never mix database, routing and AI logic in the same file. The core has no web/CLI/protocol code; the adapters (CLI, server, MCP) are thin and delegate.
4. Repositories only access the database; external HTTP calls go in services or agents.
5. **The AI layer never imports models, repositories or database connections:** all data access goes through tools (MCP).

6. **Mandatory idempotency:** every request carries a unique `request_id`; it is checked before invoking the LLM and, if it exists, the cached result is returned.
7. **Retries with exponential backoff** and a configurable limit on every external call; when exhausted, NACK to requeue or send to dead-letter.
8. **LLM provider factory:** switch provider with an environment variable, without touching business logic. A `mock` provider for all roles avoids spending in tests and load runs.
9. Scoring/rule modules are pure functions of their inputs, with no access to the database or tools, so that every firing is reproducible and auditable.
10. Log every rule firing (id, inputs, belief degree, final verdict): it is what makes the layer auditable, and it is not omitted to save writes.
11. Observability and drift detection never block or take part in the critical path: they consume already persisted logs and their failure does not affect transactions.
12. Explicitly define each component as fail-open or fail-closed: guards and judges fail closed; context retrieval and tracing fail open. A missing configuration value in a security check fails closed.
13. **Outputs as a contract:** functions return a dict and write JSON to an output folder; subagents communicate through files, not shared memory.
14. **Compact responses:** summaries by default, full data only on request (everything stays in the agent's context and is resent every turn).
15. Do not print to stdout in stdio servers: it is the protocol channel; use stderr or return values.
16. A read-only internal frontend consumes only GET endpoints and never connects directly to the database or to internal services.
17. Choose the transport deliberately and document it; a global change of transport is discussed first, and an alternative is added as an additional option.
18. **The source code is the reference:** when writing tests only `tests/` is edited. If a new test fails against the original code, the test is wrong unless there is evidence of a real bug; in that case `xfail(reason="possible bug: ...")`, never fix the source code to make it pass.
19. Work on temporary copies, never in place, when the target repo is the basis of the documented metrics.

6. Agents and LLMs

1. **In the application, the LLM never executes tools with side effects:** only deterministic code does, and every write tool is idempotent by `request_id`. GitLab Duo flows are the one exception, because writing code is their job: they run under a service account, write only to a branch of a merge request, and never edit the human-only files of section 18.10. A flow does not merge: the merge is done by the deterministic engine (`merge_guard`, section 18.9), never by an LLM's decision. What they write is not trusted until every deterministic gate of the pipeline has passed.
2. **Never trust the LLM's output:** revalidate it on the server with strict Pydantic schemas (`extra="forbid"`) and business limits.
3. The user-facing agent has minimum privilege: no access to the database, the knowledge table or the tool server. It only proposes a structured intent and, if it cannot map the message, asks instead of guessing a field.
4. A prompt injection/jailbreak filter before the message reaches the agent, which fails fast.
5. Feedback to the user exposes an objective, auditable verdict, never a judge's raw justification, which could leak internal reasoning or business rules. The agent words it but never reinterprets or overrides it.
6. Catch any exception in an agent's tool dispatch and return it to the model as an error result: one bad call must not bring down the whole process.

7. Security

1. Authenticate every tool call with a bearer token: the server stores only its SHA-256 hash, compares it in constant time and derives `client_id` from the token, never from a client header.

2. Per-tool authorization with a server-side allowlist; nothing in the prompt or in retrieved documents can widen access.
3. Rate limiting per client and tool, computed from the audit table so it holds across replicas without Redis.
4. Audit log of every invocation, including denied ones, with PII masked; if the audit write fails, the tool is not executed.
5. Mask PII in trace exports (pseudonymize user ids, redact emails and long numbers); every new PII field is added with its test before being traced.
6. One service account per service, with access per secret and never at project level.
7. Values that Terraform does not generate are loaded by hand, not through Terraform variables, so they do not end up in the state.
8. Normalize secrets (`.strip()`) on both sides of a comparison; beware of newlines in secrets.
9. Document exposure analyses (e.g. abuse of a public API) and do not implement a defense without the user's approval.
10. Persistence is opt-in and never public on routes that accept arbitrary input; do not store rounded numbers.

8. Concurrency and messaging

1. Pessimistic locking (`SELECT ... FOR UPDATE`) plus a `UniqueConstraint` on the id to prevent double processing.
2. A recovery sweeper (`FOR UPDATE SKIP LOCKED`) requeues what a crashed worker left half done.
3. Publish messages as persistent on durable queues: a transient message is lost when the broker restarts.
4. Safe ack/nack (log and continue if the channel closed) and reconnection with retry, instead of dying.
5. Long-lived consumers use `restart: unless-stopped` and wait for the broker themselves; normal services use a bounded restart as a defense.
6. Close the database transaction before long external calls (MCP, LLM).
7. Use `clock_timestamp()` and not `now()` for audit timestamps: `now()` reflects the start of a long transaction and distorts latencies.
8. In clients with a timeout, open a new session per attempt and do not reuse it across retries.
9. **In-memory state + multiple instances = bug:** locks and rate limiters must be durable (database) if the service scales horizontally.

9. Observability and traces

1. One trace per transaction, with an id derived from the `request_id`, and typed, nested observations.
2. Observation names are an API: stable, verb first, low cardinality; ids and models go in metadata. Renaming is a breaking change.
3. Tracing is fail-safe: it turns off without keys and its failures are logged without changing the result or the ACK/NACK.
4. Tests never send traces: tracing is disabled in the suite and an in-memory exporter is used when asserting on spans. If the test depends on callbacks, use real fake models and not `AsyncMock`.
5. Verify against a real trace after changing the instrumentation.

10. Evaluating models and judges

1. Evaluate the production code, never a pasted copy of the prompt.

2. The labels of the cases are decided by the user: new cases stay pending until the user reviews them.
3. Report false approvals first: they are the dangerous error and accuracy alone hides them.
4. Keep context retrieval fixed in evaluations.
5. Real runs cost money and time: use the mock provider to test the flow and pass up-to-date prices with a date.
6. Asymmetric judges from different model families; the tie-break uses a different model from the first judge's to give an independent opinion.
7. LLM verdicts vary between runs: do not repeat the vote to "correct", and cover that gap with deterministic checks.

11. Code tests

1. **Two tiers.** Locally, run only the test being written, one test or one test file at a time, to see red and green; this tier finishes in seconds and needs no external service. Everything else runs in CI: the full suite, integration and end-to-end tests, linters, type-check, mutation testing and builds.
2. **A local red is a claim; CI measures it:** the red-first check (`python -m scripts.red_check --base origin/main`) runs the tests a pull request adds against the code before the change and requires each to fail. A task of type `tests-only` or `refactor` skips it; a single test that documents behavior the code already has is declared with `@pytest.mark.characterization(reason="...")`. Both declarations are printed for the reviewer. A test that did not run on the old code fails the check; it never counts as passing.
3. CI jobs are parallelized whenever possible; they are not queued.
4. Changes in the back end launch only the back-end tests; changes in the front end, only the front-end ones.
5. Document changes do not launch tests: only what contains code.
6. Use path filters (`paths`/`paths-ignore`) in the workflows so that each job runs only when its files change.
7. Load and chaos tests (Locust, killing workers, restarting the broker) run in the cloud (manual or scheduled workflow), never locally, and do not run on every PR because of their cost and duration.

12. Code fragility and coverage (mutation testing with AST)

1. To measure fragility, **AST** mutation is used: the code is intentionally altered (mutants) and the tests are launched again. If the tests still pass, no assertion depends on that line.
2. Surviving mutants are fixed by writing or improving tests, and mutation is launched again, in a loop, until the required score is met.
3. **Mutation runs only on new or modified files**, never on files that did not change, **in parallel in CI**.
4. Validate the baseline before mutating: the suite must pass without modifying the code, or every mutant would count as "killed" and a false 100% would result.
5. **Equivalent mutants are accepted only by the human.** The agent may propose a survivor as equivalent, with its reason, in the pull request; it never writes `state/equivalent_mutants.json` (a hook denies it) and never covers the survivor with an artificial test. Each accepted entry is bound to the file's SHA-256 and expires when the file changes.
6. **Mandatory in critical modules** (money, security, concurrency, public contracts). In the rest it is optional and each project decides. A mandatory file below the minimum score blocks the PR. The gate uses K / (N − E_human) and reports the raw score K / N next to it: `python -m scripts.mutation_gate --results <engine output> --required <files>`.
7. The critical folders and the minimum score are defined in `state/config.json` (`mutation_critical_paths`, `mutation_min_score`). `python -m scripts.mutation_targets --base origin/main` lists the mandatory and optional files among the modified ones. The mutation engine and its CI job are specific to each project; they call these two scripts.

13. Verification

1. Verify after any engine change against documented baseline values, in CI.
2. **A script that says PASS does not prove it measured the right thing:** combine internal consistency (two runs agree) with a known external value.
3. **Fail loudly, do not fabricate:** if a check could not run (missing dependency, credentials), return a real error and never an empty result that looks like a pass.
4. **Fail fast:** validate credentials and configuration before spending minutes of compute.
5. Never leave fixtures or verification reference files inside the project after using them.
6. Validate with real end-to-end cases in the deployed environment, not only with tests.

14. Debugging, load and chaos

1. On a failure between containers, isolate the transport from the application: write the minimal script that exercises only that connection and run it with `docker exec` inside the failing container.
2. If the script passes, the bug is in the app's control flow or exception handling, not in Docker, DNS or the network.
3. **A missing dependency ≠ a missing config:** both look like an opaque `500`; check the server traceback before assuming.
4. Document postmortems.
5. Validate with real load (Locust) and with chaos: kill workers, restart the broker and verify invariants in SQL (zero duplicates, zero lost, nothing stuck).
6. Measure scaling by pausing the workers (`docker pause`), not with `stop`/`start`.
7. Watch out for heavy healthchecks: they can keep an idle broker at high CPU.

15. Environment and dependencies

1. **Never call `subprocess.run(["pytest", ...])` with a "bare" command name:** use `[sys.executable, "-m", ...]` to pin the interpreter and not depend on the ambient PATH.
2. Parallel checkouts/worktrees: a script installed in editable mode may point to another checkout; verify with `pip show` before trusting a run.
3. Pin exact versions of coupled packages (e.g. fastapi/starlette) and reinstall clean if they get out of sync.
4. Document environment variables in a table with description and default value.

16. Infrastructure and deployment

1. One Docker image per service, with production dependencies only and a non-root user.
2. Tag images with the commit, never with `latest`.
3. All infrastructure as code (Terraform) with versioned state.
4. Scripts to turn the demo environment off and on, to control costs.
5. Update the infrastructure documentation in the same PR as any infrastructure change.
6. Do not change broker or provider without confirming, and record each decision as an ADR (section 20).

17. When to pause and confirm

1. Confirm with the user before touching:
   - DELETE or UPDATE without WHERE.
   - Migrations that drop columns or tables.
   - The ACK/NACK logic.

- Public tools (renaming or removing) or write guards.
- Thresholds of a security guard (lowering them) or measurement operators/thresholds without re-measuring and updating the docs in the same change.
- Fixtures that affect all documented numbers.
- `.env` and secrets.
- Model, router or tool-registry files (list the impact before deleting or moving).
- Published numbers without a real measurement behind them.

2. Before a migration, verify the current revision (`alembic current`) and confirm the target with the user; this applies equally to the cloud database.
3. Hard-deny hooks protect what only the human may change: `PENDING.md` can be edited but never deleted or emptied; `state/equivalent_mutants.json` is never written by an agent; nothing under `build/` is edited; nothing is pushed to `main`. The other human-only files of section 18.10 are only checked by the `human_only_files` job of `.gitlab-ci.yml`, a weak backup control: the same merge request can edit it away, and it stops nothing on its own. The strong control (a pipeline execution policy or required code owner approval) is not in place yet, so a human reviews every change to those files.
4. When refusing an action, always state the correct alternative in the same reply.

18. Linters, branches, commits and PRs

Linters

1. Linters and type-check (`ruff`, `mypy`, and the front-end ones) may run locally on the diff for feedback in seconds; CI runs them again and its result is the one that counts.
2. They are launched only on what was modified (the diff), and only the linter for the side that changed (back end or front end).
3. They do not run on document-only changes.

Branches and commits

4. GitHub Flow: an always-deployable `main` and short branches. **Step 0, always:** before modifying any file, check the branch and the working tree (`git status` and `git branch`), because multiple agents are working on the project. Never start working on the current branch by default: first create a new, isolated branch for the task.
5. **The branch name represents the task** (`feat/`, `fix/`, `chore/` + a clear description, e.g. `feat/fuzzy-scoring-layer`).
6. **Atomic task system:** one feature = one commit with a representative name.
7. Conventional Commits: `type(scope): description`.

Pull requests

8. **Never commit or push to `main`.** Every change reaches it through a pull or merge request.
9. **Merge policy ([ADR 0007](docs/adr/0007-engine-approves-each-merge-human-approves-the-policy.md)):** this repository is hands-off. The human approves the policy once; the engine approves each merge. A merge request made by a Duo flow is merged with no person when `merge_guard` says `merge: yes`: every gate green (including `red_check`, `mutation_gate` and `security_gate`), no human-only file touched, the pipeline user and every commit author a service account, at most 3 automatic fix attempts. This is continuous deployment, on when `AFAW_AUTO_MERGE=true` (docs/autohealing.md); with it off the same verdict is only printed (continuous delivery) and a person merges. An agent never merges by another path, and the verdict is computed by code, not judged by an LLM. The classic AFAW rule that a human reviews and merges every pull request does not apply here.
10. **Human-only files:** `.gitlab-ci.yml`, `AGENTS.md`, `state/equivalent_mutants.json`, the mutation and gate settings in `state/config.json`, and `deploy/` change only in a merge re-

quest authored by a human. The automation never edits them, and the gates may never be weakened by the loop that they gate.

11. **An agent opens the merge request for its branch** without asking; the Developer and Fix CI/CD flows already do. It does not merge it (section 18.9) and it does not open one that touches a human-only file (section 18.10) unless a human asked for that change.
12. When creating a PR, deliver the title and a comment with the description in the reply, and also write them in the PR if the environment allows it.
13. The checks run in CI (`checks.yml`: task files, delta, red-first, ADRs and links, derived views; `scripts-ci.yml`: lint, types, tests); paste the link to the run in the PR, not a local output.
14. If a PR changes a measured number, update README/docs in the same PR.
15. After the merge, by the engine or by a person, delete the branch.

19. Token budget

1. Short replies: tables and lists, without repeating tool output.
2. Do not read output JSON, images or fixtures unless the task requires it; read only the part of a file that is needed and do not re-read what was just written.
3. At most 3 subagents per round, passing each one only its file and its data, not the conversation history.

20. Project knowledge: decisions and gotchas

1. **State files say what; prose says why.** Knowledge is recorded once, as markdown, never duplicated in JSON.
2. **A decision that constrains later work is an ADR** in `docs/adr/NNNN-slug.md` (template: `docs/templates/adr.md`): Context, Decision, Alternatives considered, Consequences. Record it in the same pull request that introduces it, with `status: proposed`; the human sets `accepted`.
3. **A trap others will hit is a gotcha** in `docs/gotchas/NNNN-slug.md` (template: `docs/templates/gotcha.md`): Symptom, What to do.
4. **Prefer a mechanism to a gotcha.** If a script, a configuration default, a test or a hook can prevent the trap, propose that mechanism and mark the gotcha `promoted` with `promoted_to`. A rule enforced by a mechanism no longer depends on being read.
5. A record leaves `LASTCONTEXT.md` only for an explicit reason, never for its age: an ADR is `superseded` (by one that lists it in `supersedes`) or `deprecated`; a gotcha is `resolved` (with `resolved_by`) or `promoted`. Never delete a record.
6. When the active records exceed `knowledge_budget` (in `state/config.json`), the dashboard and `LASTCONTEXT.md` say a review is due. The human prunes in a pull request; an agent never summarizes or deletes records on its own.

21. GitLab Transcend and Duo Agent Platform rules (Path B extension)

1. **Dual-forge fact collection (`collect_facts.py`):** measured facts (`started_at`, `pr_opened_at`, `first_green_at`, `merged_at`, `last_activity_at`, CI status and CI job minutes) are collected deterministically from either the GitHub API or the GitLab REST API v4 (`/api/v4/projects/:id/merge_requests` and `/pipelines`), never declared by an agent.
2. **Nine-stage GitLab DevSecOps pipeline (`.gitlab-ci.yml`):** the repository covers all nine stages: `plan`, `create`, `verify`, `secure`, `govern`, `package`, `configure`, `release` and `monitor`. Every stage runs deterministic scripts or native GitLab security and compliance checks, and fails loudly on error.
3. **Hands-off autonomy and GitLab Duo agents:** Duo Custom Agents and Flows (Developer Flow, Code Review Flow, Fix CI/CD Pipeline Flow) execute multi-step post-code workflows: they run `red_check.py`, remediate pipeline failures, verify `files_touched` with `check_`

`task_delta.py` and prepare merge requests. The human approves the policy and the engine approves each merge (section 18.9); only a human records an accepted equivalent mutant in `state/equivalent_mutants.json` or changes the human-only files of section 18.10.

4. **Service account and caps:** the flows run under the group service account, never a personal token. A merge request gets at most 3 automatic fix attempts, after which the flow comments and stops; flow runs are capped to stay inside the monthly GitLab Credits. A test is edited only if `red_check` confirms it still fails on the old code. Secrets are masked CI/CD variables. A variable that a merge request pipeline must read cannot be protected, because a protected variable is not passed to pipelines of unprotected branches: `AFAW_MERGE_TOKEN` is masked and unprotected, scoped to the least role that can merge, and created by a Maintainer or Owner (a Developer cannot). Variables only the deploy reads (`GCP_*`) stay protected.
5. **Read-only live service on Google Cloud Run:** the deployed web application serves the derived views (`build/index.html`, `LASTCONTEXT.md`, metrics) and the read-only project knowledge assistant. It never writes to the repository, never mutates state files and never paraphrases a measured number.
6. **Licence:** the MIT `LICENSE` in the root stays intact; this repository extends the base project `afaw-anti-fragile-agentic-workflow` (remote `upstream`).

B Reference repository layout

Path	Purpose
`AGENTS.md, CLAUDE.md`	Agent rules: AGENTS.md is the single source; CLAUDE.md imports it
`state/schemas/`	JSON Schemas of task files, deltas, config and equivalent mutants
`.claude/hooks/`	Session start (builds and injects LASTCONTEXT.md), safety guard, lint check
`state/config.json`	Project settings: code folders, critical paths, minimum mutation score
`state/tasks/task_NNN.json`	One task file per task, written by the agent; declared values only, never deleted
`state/equivalent_mutants.json`	Equivalent mutants accepted by the human, bound to a hash of each file; agents cannot write it
`context/tasks/task_NNN_context.json`	One delta per task, written by the agent
`scripts/collect_facts.py`	Collects measured facts per task from git and the pull-request and CI interfaces
`scripts/build_state.py`	Derives the pending view, done history, last-context views, metrics and HTML dashboard; output is never committed
`scripts/check_task_files.py`	Rejects invalid task files, declared measured values, reopened tasks, duplicate identifiers and deleted task files
`docs/adr/NNNN-slug.md`	Architecture decision records: context, decision, alternatives, consequences
`docs/gotchas/NNNN-slug.md`	Operational traps still in force, closed as resolved or promoted
`docs/templates/`	Templates for both kinds of record
`scripts/check_docs.py`	Checks records (front matter, sections, supersedes chain) and relative links
`scripts/check_task_delta.py`	Rejects a pull request whose `files_touched` differs from the diff; `--write` fills it
`scripts/red_check.py`	Runs the added tests against the merge base and requires each to fail
`scripts/mutation_targets.py`	Lists mandatory and optional files to mutate, and the minimum score
`scripts/mutation_gate.py`	Scores the engine’s results with human-accepted equivalences only
`scripts/rules_to_tex.py`	Generates this paper’s appendix from AGENTS.md
`.gitignore`	Excludes the build directory of the derived views
`.github/workflows/dashboard.yml`	Rebuilds the dashboard on push to main, hourly and on demand, and publishes it
`.github/workflows/checks.yml`	Runs the task-file and documentation checks on every pull request
`.github/workflows/`	CI with path filters and parallel jobs

Table 6. Reference layout of the boilerplate.